UTOPIA · AVION PRODUCTIONS · DEFECT REPORT

Door scanning, UPI pricing, and seven logic defects

Two reported faults — passes not scanning at the door, and the UPI QR keeping its old amount after a price edit — plus the defects found while tracing them. Every item below is fixed, on `main`, and deployed.

Repository avalreddy009-cpu/Website-for-party

Commits `5437891` … `862bb1a` (8)

Stack Next.js 16 App Router, React 19, Upstash Redis

Deployment avionproductions.vercel.app

Date 4 September 2026

Status Merged to `main`, CI and deploy green

Summary

SEVERITY	DEFECT	WHAT IT COST
HIGH	Pass QRs would not scan at the door	Staff fall back to typing codes; queue stalls
HIGH	A concurrent write could un-scan a guest	Same QR admitted twice
HIGH	Rate limits keyed off a caller-supplied header	Every limit on the site bypassable
HIGH	A URL segment used as an object key	Prototype pollution from the CMS
MEDIUM	UPI QR kept the old amount after a price edit	Buyers pay the wrong total
MEDIUM	Verification codes locked addresses out for good	Real buyers cannot check out
MEDIUM	Abandoned holds never expired	Pending list never clears; dead rows repriced
MEDIUM	Reference lookup dead for group bookings	Valid group reads as a forgery at the door
LOW	Per-tier revenue inflated on re-imported orders	Wrong numbers in the CMS
LOW	Order list could 500 on a retired pass tier	CMS page fails to load
LOW	Signed-token fields had no upper length bound	Wasted work before rejection
LOW	Pass wallet cookie could exceed the browser cap	Silently dropped; looks like a sign-out

The two reported faults

01 Pass QRs would not scan at the door HIGH

CAUSE Three independent problems stacked. The decoder ran jsQR with `inversionAttempts: "dontInvert"`, which discards light-on-dark codes — the usual result when reading off a phone screen. The payload was `UTP|{name}|{code}|{token}` with a full HMAC token, dense enough that the module size at arm's length fell below what a camera frame resolves. And a decode that produced something unrecognisable was discarded without any feedback, so the panel looked inert.

FIX New QRs carry a short `orderId.passCode.signature` token at error-correction level H. The decoder tries the platform's native `BarcodeDetector` first, then jsQR with inversion attempts on both the full frame and a centre crop. Failed camera reads now surface a red result card. QRs already emailed still verify — the old token format is parsed alongside the new one.

VERIFY 53-character payload round-trips through jsQR; door scans exercised in `scripts/e2e-check.sh`.

02 UPI QR kept the old amount after a price edit MEDIUM

CAUSE	The QR was built once, at reservation time, and the amount was frozen into it. Editing a price in the CMS changed nothing already issued. The public catalogue also fetched prices once on mount and never again, so an open landing page kept showing the old figure until reloaded.
FIX	Saving a price rebuilds the UPI QR for every unpaid hold. An open pay screen polls and picks up the new amount, and tells the buyer it changed rather than swapping the number silently. The price editor renders the live one-pass QR at the saved amount, greyed out until what you have typed is actually saved. The catalogue refetches on focus and on a slow timer.
BOUNDARY	A hold with a UTR already submitted is never repriced. What they paid is what they paid.

Defects found while tracing those

03 A concurrent write could un-scan a guest HIGH

CAUSE The store persists to Redis by writing the entire database as one blob, and every request hydrates by merging the remote copy back in. The merge replaced an order wholesale whenever the remote copy's status ranked equal or higher. Two copies both marked `paid` tie, so an instance that read before a door scan and wrote after it handed back a copy with no entry recorded — and the same QR was admitted a second time.

FIX Door entries, revoked pass codes and payment proof are treated as one-way latches during a merge, in either direction. A lost write now costs a stale name or price, not a guest walking in twice.

VERIFY `scripts/concurrency-check.sh` admits a pass, replays a pre-scan snapshot over the top, and scans again. On the old code it reported `admitted`; it now reports `already-in`.

04 Rate limits keyed off a caller-supplied header HIGH

CAUSE The limiter identified callers by the leftmost `X-Forwarded-For` entry. Anyone can set that header, so changing it per request produced a fresh counter each time and every limit on the site — staff login, one-time codes, reservations — became decorative.

FIX Prefer the headers the platform writes itself and otherwise take the last hop, the one the nearest trusted proxy appended. The counter map is also bounded now; it previously grew one entry per address per scope for the life of the process.

05 A URL segment used as an object key HIGH

CAUSE Orders live in a plain object keyed by id, and the approve and reject routes looked up the raw path segment. `POST /api/admin/orders/__proto__/approve` therefore resolved `Object.prototype`: truthy, so it passed the already-decided guard, after which `status` and `paidAt` were written onto the prototype that every object in the process inherits.

FIX Every order lookup goes through one accessor that checks own keys. Both routes validate the id the way the transfer route already did, and the Redis merge skips those keys too — assigning one there reparents the object rather than adding to it.

VERIFY Covered in `scripts/e2e-check.sh`, which asserts the prototype stays clean.

06 Verification codes locked addresses out permanently MEDIUM

CAUSE The send counter was compared against a per-window cap but only ever incremented — it never reset when the window lapsed. The fifth code an address requested was effectively its last: every later request tripped the cap for as long as they kept trying.

FIX The counter resets with the window it belongs to, and the retry-after the API reports is the real one rather than a fixed fifteen minutes.

07 Abandoned holds never expired MEDIUM

CAUSE `OrderStatus` has carried `"expired"` since the first commit and the CMS draws its badge from the hold deadline, but nothing ever set the status. A month-old abandoned hold stayed `reserved`: counted as pending, still accepting proof, and repriced on every price change. The badge and the backend disagreed.

FIX Stale unpaid holds are swept on hydrate. Submitting proof un-expires the hold rather than refusing it — someone who has already transferred money outranks a thirty-minute timer, and staff should see the row instead of receiving an email about it.

VERIFY `scripts/hold-expiry-check.sh` covers the sweep, the revival, and that a revived hold survives the next sweep.

08 Reference lookup dead for group bookings MEDIUM

CAUSE Typing a booking reference at the door only resolved orders holding a single pass. For anything larger the lookup fell through silently and the panel reported `NOT A PASS` — which reads like a forgery for a perfectly valid group.

FIX A reference admits the next unused pass on that order, so staff can walk a group through one at a time. Once all are used it reports `already-in` rather than invalid.

09 Three smaller defects LOW

- Re-imported orders stored the order total in the per-pass price field, so the CMS reported a group booking's per-tier revenue as total multiplied by quantity.
- The order summary indexed a revenue bucket by pass id without checking it existed, so a single row carrying a retired tier would throw and take the whole CMS order list down with a 500.
- The pass wallet cookie was not size-checked. Browsers drop a cookie over roughly 4 KB without reporting it, which presents identically to being signed out. It is now trimmed to fit, oldest passes first.

10 Signed-token fields had no upper bound LOW

CAUSE Token schemas set a minimum length but no maximum, so a megabyte of junk was base64-decoded and HMAC-verified before being rejected. The pass-claim route coerced its token with a bare `String()` and no check at all.

FIX Both are bounded to what the server actually mints and rejected before any cryptographic work.

Reported but deliberately unchanged

Two independent audits flagged the following as defects. On inspection both are intended behaviour, and changing either would have made the product worse.

One address can book again after re-verifying

The rule is one reservation per verification token, not one per email address. Requesting a fresh code issues a fresh token, which is the intended way to place a second order — and with a cap of eight passes per order, anyone buying more than that legitimately needs one. The harm the audits actually described, unpaid holds accumulating in the pending list, is what the hold expiry sweep in item 07 addresses.

The wrong-guess counter resets when a new code is sent

A new code is a different secret, so carrying the failed-attempt count across would only penalise people who mistyped. Total guesses stay bounded by the cap on codes per window regardless.

Rate limiting is per-process

Counters live in process memory, so on a multi-instance deployment the effective budget scales with the number of running instances. This is real but needs shared storage rather than a patch, and the code already documents it. It is noted here as known, not fixed.

Operational note

> **DO NOT REMOVE THESE ENVIRONMENT VARIABLES**
>
> `CMS_PHRASE` and `DOOR_PHRASE` are what unlock `/admin` and `/door` in production. A stale comment in the source claimed built-in hashes would take over if they were unset. They do not — staff login answers 503. The comment has been corrected. The live site currently returns 401 to a wrong phrase, confirming the variables are set and both panels unlock.

Verification

Three scripts sit in `scripts/`. They are regression tests for the defects above, not smoke tests, and each was run repeatedly against the same server to confirm it stays green.

```
node scripts/fake-upstash.mjs &    # stands in for Upstash
./scripts/dev-fixtures.sh          # dev server with test fixtures

./scripts/e2e-check.sh             # booking, price change, approval, door
./scripts/concurrency-check.sh     # the stale-snapshot double entry
./scripts/hold-expiry-check.sh     # hold sweep and late-payment revival
```

`e2e-check.sh` walks a booking from email verification through a price change, proof upload, approval and three door scans. Alongside the happy path it asserts that a paid-for amount stops moving once a UTR is in, that the CMS is never handed the QR signing material or the payment screenshot, that each panel rejects the other panels' session cookies, and that `__proto__` leaves the prototype alone.

Also confirmed: production build succeeds, lint and typecheck are clean, CI is green on all eight commits, and the live site serves its pricing endpoint with `no-store` so an edited price is never cached.